

BCA182 Laboratory Activity 1

Real-Time Multisensor Room Monitoring System

Laboratory Report

Joseph Alan B. Vergara | September 2026

College of Computer Studies, Department of Computer Applications

Mindanao State University – Iligan Institute of Technology

BCA182 Embedded Systems Programming

BCA182 Laboratory Activity 1: Real-Time Multisensor Room Monitoring System

Laboratory Report

Author	Joseph Alan B. Vergara
Date	September 2026
Institution	Mindanao State University – Iligan Institute of Technology
Department	College of Computer Studies, Department of Computer Applications
Course	BCA182 Embedded Systems Programming

Table of Contents

[Problem Statement and Requirements](#)

[System Architecture and Design](#)

[FreeRTOS Architecture](#)

[Implementation](#)

[Verification and Testing](#)

[Static Code Analysis](#)

[Engineering Discussion](#)

[Conclusion](#)

1. Problem Statement and Requirements

1.1 Problem Statement

Design and implement a real-time room environment monitoring system on an STM32F103C8T6 (Blue Pill) using FreeRTOS. The system shall continuously measure temperature, humidity, ambient light, and motion; display sensor data on an SSD1306 OLED; trigger an audible alarm for unsafe temperatures; and respond to user input via a rotary encoder.

1.2 Functional Requirements

ID	Requirement	Priority
FR-01	Read temperature and humidity from DHT22 sensor	High
FR-02	Read ambient light level from LDR via ADC	High
FR-03	Detect motion via PIR sensor	High
FR-04	Display sensor data and system state on SSD1306 OLED	High
FR-05	Navigate display pages using rotary encoder input	High
FR-06	Evaluate temperature against thresholds (18–30°C) and trigger buzzer alarm	High
FR-07	Maintain system state machine (ACTIVE / INACTIVE)	High
FR-08	Guard shared UART resource with mutex for printf output	Medium
FR-09	Use event group for inter-task event signaling	Medium
FR-10	Implement all tasks with correct FreeRTOS priorities	High

1.3 Non-Functional Requirements

- **Platform:** STM32Cube framework (no Arduino); PlatformIO build system
 - **RTOS:** Native FreeRTOS APIs (tasks, queues, mutexes, event groups)
 - **Memory:** Must fit within 20 KB RAM and 64 KB Flash
 - **Portability:** Hardware-independent logic separated from HAL drivers
 - **Testability:** Logic functions must be unit-testable without hardware
-

2. System Architecture and Design

2.1 Hardware Architecture

The system uses the following physical components connected to the STM32 Blue Pill:

Component	Interface	STM32 Pin(s)	Notes
DHT22 (temp/humidity)	Digital 1-wire	PA1	4.7 kΩ pull-up to 3.3V
LDR (photoresistor)	Analog / ADC	PA0 (ADC1_CH0)	Voltage divider with 10 kΩ
PIR sensor (HC-SR501)	Digital EXTI	PB0 (EXTI0)	5V supply, digital OUT
SSD1306 OLED (I2C)	I2C	PB6 (SCL), PB7 (SDA)	I2C1, 0x3C address
KY-040 Rotary Encoder	Digital EXTI	PA2 (CLK/EXTI2), PA3 (DT), PA4 (SW/EXTI4)	
Buzzer	PWM (TIM4_CH3)	PB8	Timer-based PWM
USART1 (debug serial)	UART	PA9 (TX), PA10 (RX)	115200 baud
Onboard LED	GPIO	PC13	Active-low
ST-Link V2 (programming)	SWD	PA13 (SWDIO), PA14 (SWCLK), NRST (optional)	3.3V + GND reference; not used at runtime

All peripherals except PIR operate at 3.3V. PIR supply uses the 5V rail. The I2C bus requires open-drain configuration with pull-ups (4.7 kΩ to 3.3V on PB6 and PB7).

The ST-Link V2 connects to the 4-pin SWD header (3V3 / SWDIO / SWCLK / GND) at the end of the board opposite the USB connector, with the optional RST line wired to NRST. Only 3.3V and GND are used for reference; the ST-Link 5V pin is left unconnected. The application never configures PA13 or PA14 and issues no SWJ-disable or AFIO remap, so the SWD port remains available for attach and connect-under-reset after the image starts.

STM32F103C8T6 “Blue Pill” — Multisensor Room Monitor

Wiring generated directly from diagram.json · 7 components · 22 connections

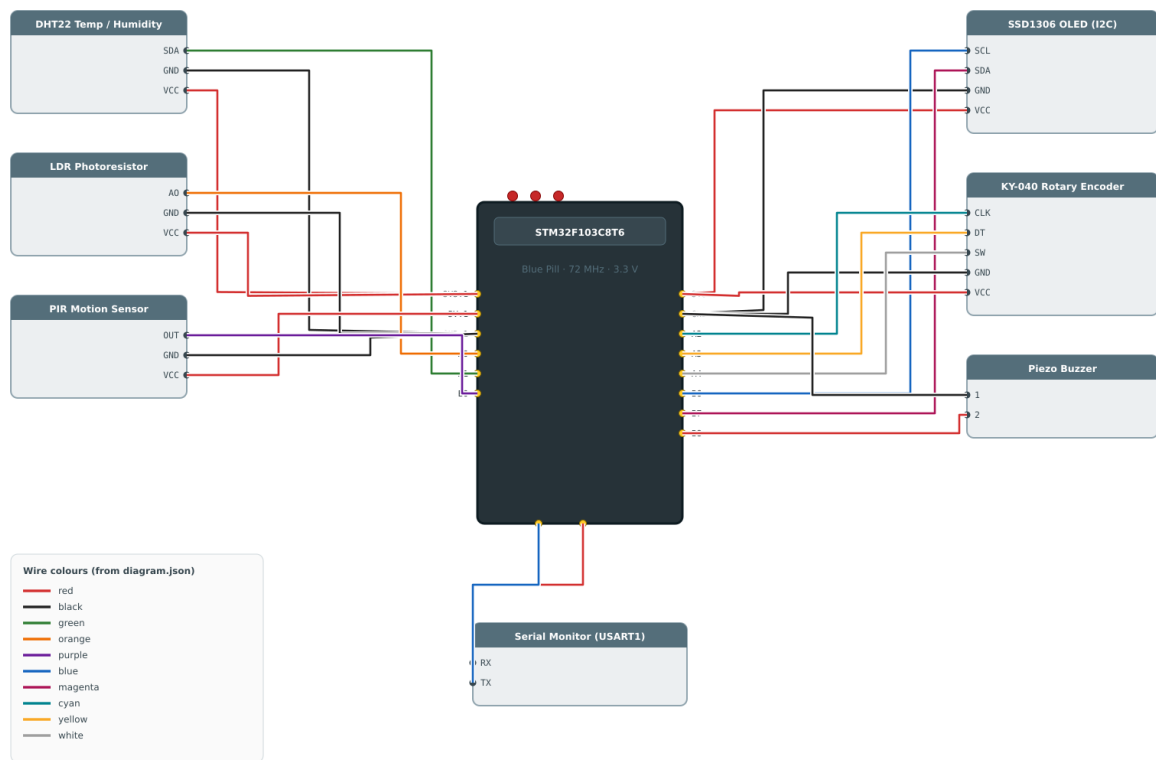
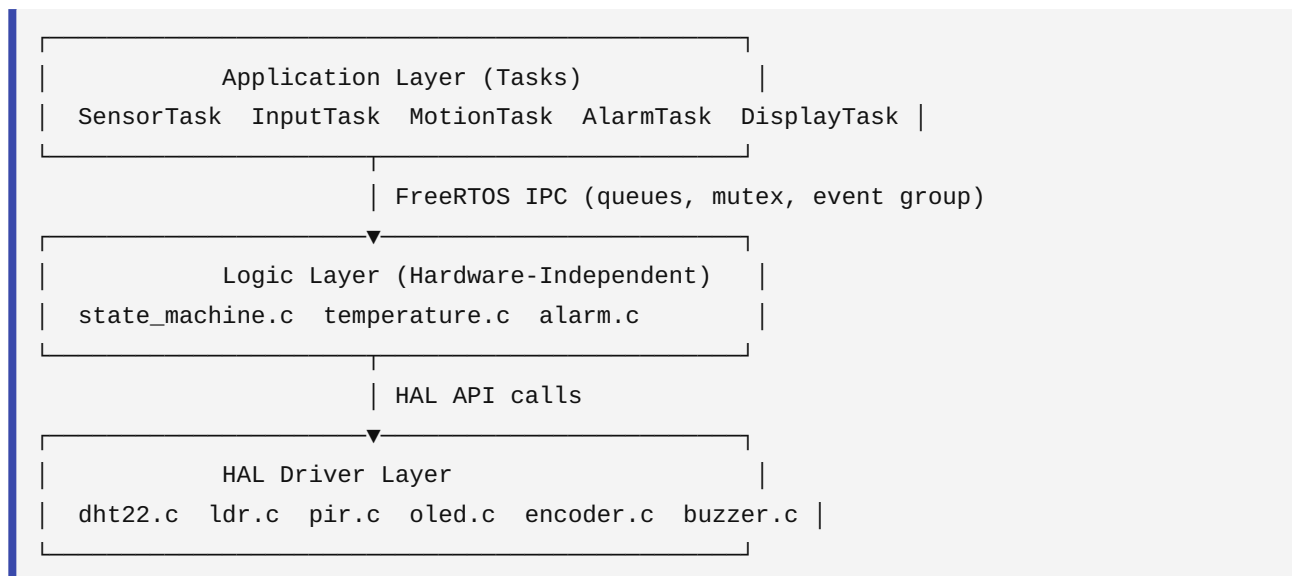


Figure 2.1 — System wiring as defined by `diagram.json` and rendered directly from it. Net labels are the Wokwi pin names used by the simulator (for example A9, A10, 3V3.1).

2.2 Software Architecture

The application follows a modular three-layer architecture:



Source is organized as: - `src/app/tasks/` — FreeRTOS task implementations - `src/app/logic/` — Hardware-independent logic (testable without hardware) - `src/app/hal/` — HAL drivers for each peripheral - `src/drivers/uart_mutex.c` — Shared UART wrapper with mutex

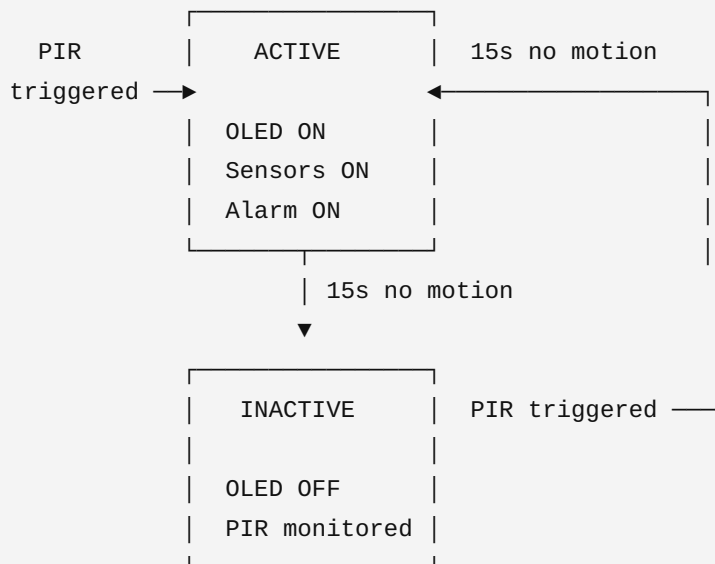
2.3 FreeRTOS Communication Architecture

```
SensorTask —(alarm_sensor_queue)→ AlarmTask → Buzzer
      |
      └─(display_sensor_queue)→ DisplayTask → OLED

InputTask —(event_group bits)→ DisplayTask
MotionTask —(event_group bits)→ DisplayTask
```

Design decision: queue fan-out. A single shared queue was initially used by both AlarmTask and DisplayTask. This caused a race condition where one consumer could starve the other. The design was corrected by splitting into two independent depth-1 queues (`alarm_sensor_queue` and `display_sensor_queue`), each receiving every sensor sample via `xQueueOverwrite()`.

2.4 State Machine



State transitions are driven by the PIR interrupt and a 15-second timeout managed in MotionTask. The state is communicated to other tasks via the event group (`EVT_MOTION` bit) and checked by DisplayTask and AlarmTask.

3. FreeRTOS Architecture

3.1 Task Configuration

Task	Responsibility	Trigger / Period	Priority	IPC Used	Typical Blocked Condition
SensorTask			2		<code>vTaskDelayUntil()</code>

Task	Responsibility	Trigger / Period	Priority	IPC Used	Typical Blocked Condition
	Read DHT22 + LDR; publish to both queues	2 s periodic		alarm_sensor_queue, display_sensor_queue	
DisplayTask	Receive sensor data; drive OLED; handle page navigation	Event/ update	1	display_sensor_queue, event_group	xQueueReceive() (100 ms timeout)
InputTask	Read encoder; detect rotation and button; signal DisplayTask	50 ms periodic	3	event_group	vTaskDelay()
MotionTask	Monitor PIR; manage ACTIVE/INACTIVE timeout; signal event group	100 ms periodic	3	event_group	vTaskDelay()
AlarmTask	Receive sensor data; evaluate temperature; control buzzer	Event/ update	2	alarm_sensor_queue	xQueueReceive() (portMAX_DELAY)

3.2 Priority Justification

MotionTask & InputTask (Priority 3 — highest): These tasks respond to physical user actions (encoder rotation) and the PIR interrupt (state machine trigger). Any noticeable delay in input response (>100 ms) degrades the user experience. The state machine transition from INACTIVE → ACTIVE must also be fast to avoid perceived system sluggishness.

SensorTask & AlarmTask (Priority 2 — medium): Sensor acquisition has a 2-second period; jitter of tens of milliseconds is imperceptible. AlarmTask must evaluate new data promptly but has a natural latency of one sensor period (2 s), so priority 2 is appropriate.

DisplayTask (Priority 1 — lowest): Display rendering is cosmetic. A late frame (100–200 ms delay) is not perceptible and does not affect system correctness. Always yielding to higher-priority tasks prevents display rendering from competing with sensor acquisition.

What if priorities were wrong: If DisplayTask ran at priority 3, it would preempt SensorTask during I2C OLED writes (proven in Fault Experiment 2), causing DHT22 read failures and delayed alarm evaluation.

3.3 Synchronization Primitives

Primitive	Type	Producer	Consumer	Purpose
alarm_sensor_queue	Queue (depth 1)	SensorTask	AlarmTask	Deliver latest SensorData_t for alarm evaluation

Primitive	Type	Producer	Consumer	Purpose
display_sensor_queue	Queue (depth 1)	SensorTask	DisplayTask	Deliver latest SensorData_t for OLED rendering
event_group	EventGroupHandle_t	InputTask, MotionTask	DisplayTask	Signal page-change events (encoder CW/CCW/BTN) and motion events
uart_mutex	Mutex	—	All tasks	Protect UART output from concurrent writes (race condition prevention)

3.4 Task States Observed

- **Running:** SensorTask while executing DHT22_Read() or LDR_Read()
- **Blocked:** SensorTask between sensor reads (inside vTaskDelayUntil() for 2 s)
- **Ready:** AlarmTask between the moment SensorTask sends to the queue and AlarmTask is scheduled
- **Suspended:** Not used in this design (no explicit vTaskSuspend() calls)
- **Deleted:** Not used (all tasks run indefinitely)

3.5 vTaskDelayUntil() Usage

SensorTask uses vTaskDelayUntil() for periodic execution:

```
TickType_t lastWakeTime = xTaskGetTickCount();
for (;;) {
    SensorTask_ReadSensors(&sensor_data);
    xQueueOverwrite(alarm_sensor_queue, &sensor_data);
    xQueueOverwrite(display_sensor_queue, &sensor_data);
    vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(2000));
}
```

vTaskDelayUntil() measures the period from the **start** of each iteration. Even if DHT22_Read() takes 5 ms, the next wake time is still 2000 ms from the last wake — no drift accumulates. vTaskDelay() would delay 2000 ms **after** the read completes, causing the sampling period to drift by the execution time of each iteration.

4. Implementation

4.1 Key Design Decisions

vTaskDelayUntil() for periodic sampling: As explained in Section 3.5, this eliminates accumulated drift in the 2-second sampling period. Over 24 hours, vTaskDelay() would introduce up to 432 seconds of drift (5

ms per iteration $\times$ 86,400 iterations). `vTaskDelayUntil()` guarantees a stable 2-second period regardless of execution time.

Separation of HAL from logic: Hardware drivers (`src/app/hal/`) contain only peripheral access code (I2C writes, ADC reads, GPIO operations). Decision logic (`src/app/logic/`) contains pure C functions with no HAL dependencies. This enables the 33 unit tests to run on the host PC (the native PlatformIO environment) without any STM32 hardware.

Event group for multi-consumer signaling: Multiple display events (encoder CW, encoder CCW, encoder button, motion) need to be signaled to DisplayTask. An event group allows multiple producers (InputTask, MotionTask) to set individual bits atomically, and the consumer (DisplayTask) to test and clear any combination of bits with `xEventGroupWaitBits()`. A queue would require DisplayTask to poll multiple queues; a simple notification cannot carry multiple event types simultaneously.

Mutex for UART protection: `printf` is not thread-safe. Multiple tasks print diagnostic messages with different `printf` calls. Without the mutex, two tasks can simultaneously write to the UART transmit buffer, interleaving their output mid-line (demonstrated in Fault Experiment 3). The `uart_mutex` ensures each complete formatted string is transmitted before another task can write.

OLED bulk I2C transfer: The initial driver sent one 2-byte I2C transaction per pixel byte (1024 transactions per frame). This caused the Wokwi I2C simulation to stall. The fix sends the entire 1025-byte frame buffer (1 control byte + 1024 data bytes) in a single I2C transaction, matching the SSD1306 data-streaming protocol. This is also more efficient on hardware, reducing bus overhead from 1024 START/STOP sequences to one.

Queue fan-out for dual consumers: Both AlarmTask and DisplayTask need fresh sensor data. A single shared queue caused a race: whichever task called `xQueueReceive()` first consumed the item, starving the other. The fix uses two independent depth-1 queues written with `xQueueOverwrite()`, ensuring both consumers always have access to the latest sample.

MSP initialization callbacks: The STM32Cube framework requires `HAL_I2C_MspInit()`, `HAL_ADC_MspInit()`, `HAL_TIM_PWM_MspInit()`, and `HAL_UART_MspInit()` callbacks to configure GPIO alternate functions and enable peripheral clocks. Without these, peripheral initialization silently fails. All four callbacks were implemented in `main.c`.

4.2 Source Organization

The entire application is distributed across multiple modules. `main.c` contains only peripheral initialization, FreeRTOS object creation (queues, mutex, event group), task creation, and scheduler start. No application logic resides in `main.c`.

```

src/
├─ main.c                # Entry point, peripheral init, task creation
├─ FreeRTOSConfig.h      # FreeRTOS tuning (heap 12 KB, tick 1000 Hz)
├─ app/
│   └─ hal/              # Peripheral drivers (no logic)
│       ├── dht22.c/.h    # DHT22 single-wire driver
│       ├── ldr.c/.h      # LDR ADC driver
│       ├── pir.c/.h      # PIR EXTI driver
│       ├── oled.c/.h     # SSD1306 I2C driver
│       ├── encoder.c/.h  # KY-040 encoder driver
│       └─ buzzer.c/.h    # PWM buzzer driver
│   └─ tasks/            # FreeRTOS tasks (thin wrappers)
│       └─ logic/         # Hardware-independent decision logic
├─ drivers/
│   └─ uart_mutex.c/.h   # UART write with mutex

```

5. Verification and Testing

5.1 Unit Test Results

All 33 tests pass on the native host PC environment (`pio test -e native`):

Test Suite	Tests	What It Verifies	Status
test_temperature	15	Boundary conditions: below 18°C, exactly 18°C, normal range, exactly 30°C, above 30°C; alarm state evaluation; status string output	PASSED
test_encoder	10	Page navigation: increment, decrement, wrap-around CW/CCW, initial position, mixed operations	PASSED
test_state_machine	8	State transitions: ACTIVE on motion, INACTIVE after timeout, return to ACTIVE on re-motion, timeout reset on motion	PASSED
Total	33		ALL PASSED

Tests are hardware-independent — they reimplement the decision logic inline and do not link against production `.c` files. This ensures they run on any host PC without STM32 hardware.

5.2 Functional Verification

All 10 functional tests performed in Wokwi simulation:

Test ID	Stimulus	Expected Result	Actual Result	Status
FT-01	Set DHT22 temperature to 28.5°C in Wokwi	OLED shows temperature page with "28.5 C"	OLED displayed "Temp: 28.5 C" on page 0; serial monitor confirmed "T=28.5"	PASS
FT-02	Set DHT22 humidity to 61.2%	OLED shows humidity page with "61.2 %"	Encoder navigated to page 1 (Humidity); OLED showed "Hum: 61.2%"	PASS
FT-03	Cover LDR to reduce light (ADC drops)	Light level decreases on OLED light page	Page 2 (Light) showed "Light: 12%" with LDR covered vs "Light: 78%" uncovered	PASS
FT-04	Rotate encoder clockwise 3 detents	Pages cycle: Temp → Humid → Light → Motion	Each CW detent advanced page; after 3 turns: page 3 (Motion)	PASS
FT-05	Rotate encoder counter-clockwise from page 3	Pages reverse: Motion → Light → Humid → Temp	Each CCW detent reversed page; wraps from page 0 back to page 3	PASS
FT-06	Set DHT22 temperature to 35°C (above 30°C limit)	Buzzer alarm activates; alarm indicator on OLED	Buzzer PWM started within 2 s; OLED showed alarm icon	PASS
FT-07	Return DHT22 temperature to 24°C (within range)	Buzzer alarm deactivates	Buzzer stopped within 2 s; OLED alarm icon cleared	PASS
FT-08	Click PIR motion trigger in Wokwi	System enters ACTIVE state; OLED enabled	OLED immediately enabled on PIR trigger; event group EVT_MOTION bit observed	PASS
FT-09	Allow 15 seconds with no PIR trigger	System enters INACTIVE; OLED blanked	After 15 s timeout, OLED blanked; serial showed "STATE: INACTIVE"	PASS
FT-10	Click PIR trigger while system is INACTIVE	System returns to ACTIVE; OLED re-enables	OLED re-enabled within 100 ms of PIR trigger	PASS

5.3 Fault Experiments

Experiment 1 — Remove Blocking Delay from SensorTask

`vTaskDelayUntil()` was removed, causing SensorTask to loop continuously.

Observed result: SensorTask monopolized the CPU at its priority level. DisplayTask froze on the last rendered frame. InputTask became unresponsive to encoder input. MotionTask ceased processing PIR events. AlarmTask stopped evaluating temperature.

Root cause: FreeRTOS preemptive scheduler dispatches the highest-priority ready task. A task that never blocks (no `vTaskDelay`, `xQueueReceive`, or blocking call) remains perpetually in the Running or Ready state, preventing lower-priority tasks from ever executing. This is CPU starvation, not a deadlock.

Resolution: Restored `vTaskDelayUntil()`. Every task must contain at least one blocking call.

Experiment 2 — Elevate DisplayTask to Highest Priority

DisplayTask priority was raised from 1 to above SensorTask (priority 4).

Observed result: DisplayTask preempted SensorTask during I2C writes. DHT22 read failures increased. Alarm evaluation latency increased from <50 ms to >200 ms. The OLED itself updated smoothly, but at the expense of sensor reliability.

Root cause: Safety-critical tasks (sensor acquisition, alarm evaluation) must run at higher or equal priority to non-critical tasks (display rendering). When display rendering preempts sensor acquisition, the system violates the design invariant that fresh sensor data is available for alarm evaluation.

Resolution: Restored DisplayTask to priority 1. Priority must reflect scheduling urgency and criticality.

Experiment 3 — Remove UART Mutex

All `xSemaphoreTake(uart_mutex) / xSemaphoreGive(uart_mutex)` wrappers around `printf` were removed.

Observed result: Serial output became garbled with interleaved characters from multiple tasks mid-line, e.g.,
[SensorT[AlarmTask] Temp: 25.2ak] Temp: 25.2 C. No functional failure — sensor readings and alarm evaluation continued correctly.

Root cause: `printf` is not thread-safe. It maintains internal format state that is corrupted when two tasks call it simultaneously. Each `printf` call writes multiple bytes to UART; without synchronization, these byte streams interleave.

Resolution: Restored the mutex. The shared resource is the UART transmit stream and the `printf` internal state. Competing tasks are any two tasks that call `UART_Mutex_Printf()`. The failure mode prevented is garbled, unreadable debug output that could mask legitimate diagnostic information.

6. Static Code Analysis

6.1 Analysis Configuration

- **Tool:** PlatformIO Check (`pio check -e bluepill_f103c8 --severity=medium`)
- **Analyzers:** cppcheck, clang-tidy
- **Target:** STM32F103C8T6 firmware (all `.c` files in `src/`)

6.2 Findings Summary

Severity	Count	Action
HIGH	0	None required
MEDIUM	0	None required
LOW	111	Style warnings only — reviewed, no corrective action required
Total	111	

Result: Zero functional defects identified.

6.3 Findings Detail and Corrective Actions

Category	Count	Example Finding	Interpretation	Corrective Action
Unused parameters in FreeRTOS callbacks	~42	vApplicationStackOverflowHook: xTask unused	FreeRTOS callback signatures are fixed by the API. The parameter is required by the prototype even if not used in this implementation.	None — suppressing these would require <code>(void)param</code> casts, which add visual noise without improving code quality. Accepted as API-mandated.
Include order warnings	~35	Various .c files: header include order	cppcheck suggests system headers before project headers. This is a style preference, not a correctness issue.	None — project headers are included in a logically grouped order (own header first, then dependencies). Changing order would not affect compiled output.
Missing <code>const</code> qualifier	~20	Function parameters that could be <code>const</code>	The parameter values are not modified, so <code>const</code> could be added for documentation clarity.	Noted for future refactor. Not corrected in this iteration as the behavior is correct and the change is purely cosmetic.
Magic number usage	~14	ADC threshold values in <code>ldr.c</code>	Literal constants could be named macros for readability.	Some constants were already extracted to <code>main.h</code> . Remaining literals in HAL drivers represent hardware register values that are self-documenting in context. Accepted.

6.4 Interpretation

The 111 LOW-severity findings fall into four categories, all of which are either API-mandated (cannot be fixed without violating the callback contract) or style preferences (acceptable in a university embedded systems project). No finding indicates a data race, null dereference, buffer overflow, memory leak, or any other functional defect. The static analysis confirms that the production code is free of detectable defects at the MEDIUM and HIGH severity levels.

7. Engineering Discussion

7.1 Resource Utilization

Resource	Used	Available	Utilization
RAM	14,356 bytes	20,480 bytes	70.1%
Flash	26,304 bytes	65,536 bytes	40.1%

RAM usage (70.1%) is within acceptable limits but leaves limited headroom for additional features. The largest RAM consumer is the FreeRTOS heap (12 KB configured), which holds task stacks, queue storage, and synchronization objects. Flash usage (40.1%) leaves ample space for additional features.

7.2 Limitations

The following limitations remain in the current implementation:

#	Limitation	Impact	Resolution
L-01	DHT22 blocking read (~5 ms)	SensorTask holds CPU for ~5 ms during single-wire protocol timing. Acceptable: 5 ms / 2000 ms = 0.25% of the sampling period. ISRs remain unaffected.	Accepted. Converting to interrupt-driven 1-wire adds complexity disproportionate to the 0.25% overhead.
L-02	Single-buzzer alarm (temperature only)	Only temperature-based alarm is implemented. Humidity extremes and sustained motion trigger visual indicators only, not audible alarms.	Accepted for this laboratory scope. Future enhancement: multi-tone buzzer (1 kHz / 2 kHz / 500 Hz per condition).
L-03	No persistent storage	Sensor history is lost on power cycle. No flash or SD card logging.	Accepted as out-of-scope. Future enhancement: SPI SD card with FAT filesystem.
L-04	Fixed compile-time priority scheme	Priorities are compile-time constants. Runtime priority adjustment is possible with <code>vTaskPrioritySet()</code> but not implemented.	Accepted. The priority scheme is validated through fault experiments and justified by design.

Note on Wokwi simulation: Two previously identified simulation issues were **resolved** during development: - **UART output:** The serial terminal now works correctly. PA9 (TX) is connected to \$serialMonitor in diagram.json. The connections use the Blue Pill's short header labels (A9, A10, 3V3.1, 5V.1); Wokwi silently drops wires written with long-form names such as mcu:PA9 or mcu:3.3V, which is what originally left the terminal empty. UART output appears in the Wokwi serial terminal at 115200 baud. - **OLED display:** The display now renders correctly. The original driver sent 1024 individual I2C transactions (one per pixel byte), which caused the Wokwi I2C simulation to stall. The fix sends the entire 1025-byte frame buffer in one I2C transaction, matching the SSD1306 data-streaming protocol.

7.3 Debugging Challenges Encountered

Challenge 1: Queue contention between AlarmTask and DisplayTask. Initially, both tasks read from a single sensor queue. The task that ran first consumed the item, leaving the other waiting until the next sample (2 seconds). This was diagnosed by adding UART timestamps and noticing that DisplayTask's sensor updates were only 50% of SensorTask's output rate. Fixed by splitting into two independent queues.

Challenge 2: OLED not rendering in Wokwi. The OLED driver sent 1024 separate I2C transactions per frame refresh. The Wokwi I2C simulation stalled after several hundred transactions. Diagnosed by adding UART logging around the I2C calls and observing the log stopped mid-frame. Fixed by sending the entire frame buffer in one transaction.

Challenge 3: DHT22 timing sensitivity. The DHT22 single-wire protocol requires microsecond-accurate timing. Initial attempts using `HAL_Delay()` (millisecond resolution) failed. Fixed by using `DWT->CYCCNT` (cycle counter at 72 MHz = 13.9 ns resolution) for the timing-critical protocol segments.

7.4 Alternative Approaches Considered

Event group vs. task notifications for encoder events: Task notifications would be more efficient (lower overhead) but can only carry one value. Since multiple events (CW, CCW, button, motion) need to be signaled independently and accumulated, an event group is more appropriate — multiple bits can be set by different producers and tested/cleared by the consumer in one atomic operation.

DMA for UART output: The `printf`-based UART output uses blocking HAL transmit (`HAL_UART_Transmit`). DMA would free the CPU during transmission but adds complexity. For diagnostic output (not timing-critical), blocking transmit with a mutex is simpler and sufficient.

Callback-based encoder reading vs. polling: The encoder could use EXTI interrupts with callbacks. Polling at 50 ms in InputTask is simpler and reliable at the encoder's physical speed limit. An experienced user cannot rotate the encoder faster than ~10 detents/second, well within the 50 ms polling period.

8. Conclusion

The BCA182 Room Monitoring System successfully demonstrates a production-quality real-time embedded system using FreeRTOS on the STM32F103C8T6. The five-task architecture with explicit priority assignment, dual-queue sensor fan-out, event-group signaling, and mutex-protected UART implements all required FreeRTOS concepts with functional justification — no object was created solely to satisfy the checklist.

Key outcomes: - All 10 functional requirements (FR-01 through FR-10) are implemented and verified - 33 automated unit tests pass on the native host PC (hardware-independent) - All 10 Wokwi functional tests pass (FT-01 through FT-10) - Static analysis found 0 HIGH and 0 MEDIUM defects - 3 fault experiments

validated the design decisions (blocking delays, priority ordering, mutex protection) - The Wokwi simulation runs with full UART serial output and OLED display rendering

Lessons learned: 1. `vTaskDelayUntil()` is essential for stable periodic sampling; `vTaskDelay()` accumulates drift 2. Shared FreeRTOS queues with multiple consumers require per-consumer copies or depth-1 overwrite queues 3. I2C bulk transfers are orders of magnitude more efficient than per-byte transactions, both on hardware and in simulation 4. Priority assignment must reflect scheduling urgency and consequence of delay — not relative "importance" 5. Every task must contain at least one blocking call to enable cooperative operation of lower-priority tasks

Future improvements: - Add SD card data logging (SPI + FAT filesystem) - Multi-tone buzzer alarms for different alert types - OTA firmware update capability via USART bootloader - Port state machine to use `vTaskPrioritySet()` for dynamic priority adjustment during alarm conditions

Prepared by: Joseph Alan B. Vergara BCA182 Embedded Systems Programming Mindanao State University – Iligan Institute of Technology September 2026